Step 2 — Customer Lookups That Run at the Same Time — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — What the prompt needs, and where it comes from
- Hint 2 — The plan, and almost the whole thing

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — What the prompt needs, and where it comes from

The billing and technical prompts get two new slots: `{account}` and `{orders}`.

Something in front of the prompt has to produce a dict with **exactly** the keys the prompt uses: `ticket`, `account`, `orders`.

Two fake lookup functions fill `account` and `orders`. Each one calls `time.sleep(1)` on purpose — without the wait, you can't see any difference between "one after another" and "at the same time".

A plain function in front of a chain becomes a step of its own:

```
look_up_customer | prompt | model | parser
```

`look_up_customer(inputs)` returns `{"ticket": ..., "account": ..., "orders": ...}` — so its output fits the prompt directly.

Leave `general` without lookups: a general question needs no account data.

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Doc05's LCEL topic: "a `dict` becomes a step that runs each value in parallel (`RunnableParallel`)".

```
RunnableParallel({
    "ticket":  get_ticket_text,     # returns inputs["ticket"]
    "account": get_account_info,
    "orders":  get_order_history,
})
```

Each function gets the **same** input dict. They run at the same time, and the result is a dict with the same keys.

Prove it, don't claim it — `scripts/time_lookups.py` :

```
start = time.time(); run both lookups one by one; one_by_one = ...
start = time.time(); build_lookup_step().invoke(inputs); parallel = ...
print both
```

You should see about 2.0 s vs. 1.0 s. Put the same check in `tests/unit/test_lookups.py` (`seconds < 1.5`), so a later change can't quietly undo it.

Full solution: Show me the solution